EfficientNet_B0_Kas_ROI_Kod_Calisma_Mantigi_Teknik_Raporu.pdf - local repository text copy

AISC DeepFake Çalışmaları | EfficientNet-B0 Kaş ROI Kod Raporu
Nazlıcan / Deney 1 - Teknik Kod Çalışma Mantığı Raporu
EFFICIENTNET-B0 KAŞ ROI
DEEPFAKE TESPİT NOTEBOOKU
Kod Çalışma Mantığı, Veri Hattı, Eğitim Mimarisi ve Kalite Kapıları Teknik
Raporu
Proje: AISC DeepFake Çalışmaları
Deney: Nazlıcan / Deney 1 / Kaş
Notebook: EfficientNet_B0_Eyebrow_Nazlican_FINAL_STABLE (1).ipynb
Model: Torchvision EfficientNet-B0 - Transfer Learning
Bölge: Kaş ROI (Eyebrow Region of Interest)
Rapor Tarihi: 08 Ağustos 2026
Rapor kapsamı: Bu belge yalnızca yüklenen notebookun gerçek kod akışını açıklar. Kodda bulunmayan bir
veri
işleme, eğitim veya değerlendirme mekanizması varmış gibi gösterilmemiştir.
AISC DeepFake Çalışmaları | EfficientNet-B0 Kaş ROI Kod Raporu
Nazlıcan / Deney 1 - Teknik Kod Çalışma Mantığı Raporu
İçindekiler
1. Yönetici Özeti
2. Kodun Temel Amacı ve Kapsamı
3. Uçtan Uca Çalışma Akışı
4. Konfigürasyon ve Deney Parametreleri
5. Hücre Bazlı Ayrıntılı Kod Analizi
6. Veri Doğrulama, Muhasebe ve Leakage Kontrolleri
7. Kaş ROI Ön İşleme Tasarımı
8. Dataset ve DataLoader Mimarisi
9. EfficientNet-B0 Model Mimarisi
10. Loss, Metrikler ve Threshold Seçimi
11. Checkpoint, Resume ve RNG Yönetimi
12. Eğitim Döngüsü ve İki Aşamalı Transfer Learning
13. Kalite Kapıları ve Hata Yönetimi
14. Final Test, Tahmin Kayıtları ve Görselleştirme
15. Deney Çıktı Dosyaları ve Manifest
16. Tekrar Üretilebilirlik ve Sürdürülebilirlik
17. Kodun Güçlü Yönleri
18. Kodun Sınırları ve Dikkat Edilecek Noktalar
19. Sonuç
Ek A. Hücre Haritası
Ek B. Kritik Fonksiyonlar ve Görevleri
AISC DeepFake Çalışmaları | EfficientNet-B0 Kaş ROI Kod Raporu
Nazlıcan / Deney 1 - Teknik Kod Çalışma Mantığı Raporu
1. Yönetici Özeti
Bu notebook, önceden kırpılmış kaş ROI görüntülerini kullanarak real/fake ikili deepfake sınıflandırması yapan, ImageNet üzerinde önceden eğitilmiş EfficientNet-B0 modelini iki aşamalı transfer learning yaklaşımıyla eğiten bir Google Colab deney hattıdır. Kod yalnızca model eğitimini gerçekleştirmez; veri doğrulama, deney izlenebilirliği, atomik dosya yazımı, checkpoint doğrulaması, tekrar üretilebilirlik, split
sızıntısı kontrolleri, threshold seçimi, final test ve çıktı manifesti üretimini de aynı deney akışı içinde yönetir.
Kodun kaş bölgesine özgü en önemli uyarlaması, kaş ROI görüntülerinin geniş ve kısa geometrisini bozmamak için doğrudan 224x224 esnetme yerine en-boy oranını koruyan resize + padding uygulanmasıdır. Böylece kaş şekli ve yatay geometrisi mümkün olduğunca korunurken EfficientNet-B0 modelinin beklediği kare giriş boyutu sağlanır.
Eğitim iki aşamalıdır: ilk aşamada EfficientNet-B0 feature extractor kısmı dondurulur ve yalnızca yeni ikili
sınıflandırıcı eğitilir; ikinci aşamada bütün ağ açılarak daha düşük öğrenme oranıyla fine-tuning yapılır.
Model seçimi validation ROC-AUC üzerinden yapılır. Karar threshold değeri test setinden değil, validation

setinde F1 ve balanced accuracy kriterleriyle seçilir. Test seti yalnızca final değerlendirmede kullanılır.
En kritik metodolojik davranış: Notebook upstream train/val/test ayrımını yeniden frame düzeyinde bölmez.

source_video/person_id bilgisi yoksa bunu uydurmaz; path ve input SHA-256 kesişimlerini kontrol eder ve gerçek
video/kişi izolasyonunu NOT_VERIFIABLE olarak kaydeder.
2. Kodun Temel Amacı ve Kapsamı

Notebookun ana görevi, aşağıdaki sabit proje yapısında bulunan kaş ROI görüntülerini güvenli ve izlenebilir
biçimde kullanarak EfficientNet-B0 tabanlı bir deepfake sınıflandırma deneyi yürütmektir:

```
Deney1_Source/AISC DeepFake Çalışmaları/Deneyler/Nazlıcan/Deney 1/
├── Kaş/
│   ├── metadata.csv
│   ├── real/{train,val,test}/...
│   └── fake/{train,val,test}/...
└── Sonuçlar/<run_id>/...
```

Girdi klasörü olan Kaş dizini bu notebook açısından salt okunur bir veri kaynağı olarak ele alınır. Kod eğitim
sırasında kaş görüntülerini yeniden kırpmaz, yeni ROI üretmez ve kaynak klasördeki görselleri değiştirmez.
Tüm yeni dosyalar yalnızca Sonuçlar altındaki run dizinine veya geçici Colab lokal cache dizinine yazılır.
Sınıflandırma hedefi ikilidir: real sınıfı 0 (negative), fake sınıfı 1 (positive) olarak kodlanır. Model tek logit
üretir ve eğitimde BCEWithLogitsLoss kullanılır.

3. Uçtan Uca Çalışma Akışı

Adım	Bileşen	Kodun yaptığı işlem
1	Ortam	Kütüphaneleri yükle, Python/PyTorch/CUDA durumunu raporla.
2	Konfigürasyon	Seed, epoch, LR, threshold aralığı, checkpoint ve cache ayarlarını sabitle.
3	local source + yollar	local source package bağla; Nazlıcan/Deney 1/Kaş ve Sonuçlar yollarını doğrula.
4	Run yönetimi	Yeni run_id oluştur veya uyumlu ve tamamlanmamış runı resume et.
5	Audit	dosyaları config, environment, pip freeze ve log dosyasını atomik olarak üret.

Adım	Bileşen	Kodun yaptığı işlem
6	Metadata	Şema, durum, label, split, sample_id, dosya varlığı ve leakage kontrollerini yap.
7	Lokal cache	Uygun kaş ROI görsellerini Colab SSD'ye doğrulayarak kopyala.
8	Preprocessing	Aspect ratio koru, 224x224 canvas'a pad et, train-only augmentation ve ImageNet normalize uygula.
9	Dataset/Loader	Train/val/test Dataset ve DataLoader nesnelerini oluştur.
10	Model	Pretrained EfficientNet-B0 yükle; classifierı Dropout + tek çıkışlı Linear katmanla değiştir.
11	Quality gates	I/O speed, 2-batch forward/backward smoke ve checkpoint round-trip testlerini çalıştır.
12	Stage 1	Backbone frozen, classifier eğitimini yürüt.
13	Stage 2	Stage 1 best ağırlıklarından tüm ağı düşük LR ile fine-tune et.
14	Validation	Best fine-tune checkpoint ile threshold ara ve validationdan seç.
15	Test	Seçilen threshold ile test setini bir kez değerlendir; tahminleri kaydet.
16	Raporlama	Loss/metric/ROC/PR/confusion/threshold figürlerini üret.
17	Reload testi	best.ckpt sıfırdan modele yüklenip inference verebiliyor mu doğrula.
18	Kapanış	run_summary, RUN_COMPLETE ve output_manifest oluştur.

4. Konfigürasyon ve Deney Parametreleri

Parametre	Değer	İşlev
seed	42	Python, NumPy, PyTorch ve DataLoader tekrar üretilebilirliği.
image_size	224	EfficientNet-B0 giriş canvas boyutu.
batch_size	32	Her iterasyonda işlenen örnek sayısı.
num_workers	0	DataLoader worker sayısı; kodda varsayılan 0.

frozen_epochs 5 Stage 1 azami epoch sayısı.
finetune_epochs 15 Stage 2 azami epoch sayısı.
frozen_lr 1e-3 Yalnız classifier eğitilirken kullanılan
başlangıç LR.
finetune_lr 2e-5 Tüm ağ fine-tune edilirken kullanılan
başlangıç LR.
weight_decay 1e-4 AdamW L2-benzeri düzenlileştirme
katsayısı.

Parametre Değer İşlev
early_stopping_patience 4 Val monitor iyileşmezse stage'i erken
bitirme eşiği.
scheduler_patience 2 ReduceLROnPlateau bekleme süresi.
grad_clip_norm 1.0 Gradient norm clipping üst sınırı.
epoch_checkpoint_keep 3 Döngüsel epoch checkpoint sayısı.
dropout 0.2 Yeni classifier öncesi dropout.
threshold_min / max 0.05 / 0.95 Validation threshold arama sınırları.
threshold_steps 181 0.005 aralıkla 181 aday threshold.
accepted_statuses ok, success Eğitime uygun metadata durumları.
negative / positive real / fake 0 = real, 1 = fake.
cache_images_locally True local source yerine Colab SSD'den eğitim.
verify_cached_images True Cache dosyalarını PIL verify ile kontrol.
resume_compatible_run True Uyumlu tamamlanmamış runı sürdür.
allow_unverifiable_source_split True source_video yoksa uyarıyla eğitime izin
verir.
Tüm parametreler frozen=True dataclass içinde tutulur. Böylece deney sırasında CONFIG alanlarının yanlışlıkla değiştirilmesi engellenir; config_resolved.yaml dosyası ile çalıştırılan konfigürasyon ayrıca

kalıcılaştırılır.
5. Hücre Bazlı Ayrıntılı Kod Analizi
Cell 1 - Imports
Python standart kütüphaneleri, veri işleme (NumPy/Pandas/PIL), modelleme (PyTorch/Torchvision), metrik (scikit-learn), YAML, Matplotlib ve tqdm yüklenir. ImageFile.LOAD_TRUNCATED_IMAGES=False ile bozuk/truncated görsellerin sessizce kabul edilmesi engellenir. Çalışma ortamında Python, PyTorch, CUDA ve varsa GPU adı yazdırılır.
Cell 2 - Configuration
RUN_SCHEMA_VERSION ve frozen Config dataclass tanımlanır. Eğitim, cache, threshold, checkpoint, early stopping ve split politika parametrelerinin tek merkezden yönetilmesini sağlar. DEVICE otomatik olarak
cuda veya cpu seçilir.
Cell 3 - Reproducibility
PYTHONHASHSEED, random, NumPy ve Torch seedleri 42'ye sabitlenir. CUDA seedleri ayarlanır. torch.use_deterministic_algorithms(True, warn_only=True) ve cuDNN deterministic seçenekleri açılır; benchmark kapatılır.
Cell 4 - Mount local source package
Notebookun Colab için tasarlandığını doğrular, Deney1_Source_Runtime konumuna local source bağlar ve LocalSource dizininin
gerçekten oluştuğunu kontrol eder.
AISC DeepFake Çalışmaları | EfficientNet-B0 Kaş ROI Kod Raporu
Nazlıcan / Deney 1 - Teknik Kod Çalışma Mantığı Raporu
Cell 5 - Fixed Project Paths
Deney 1, Kaş, metadata.csv ve Sonuçlar yollarını oluşturur. Kaş/real, Kaş/fake ve metadata.csv eksikse eğitim başlamadan hata verir. Sonuçlar dizinini gerekirse oluşturur.
Cell 6 - Run Directory + Safe Resume
SSOT biçiminde tarih/saat + eyebrow_efficientnet_b0_seed42 run ID üretir. Yalnız schema version ve suffix

eşleşen, RUN_COMPLETE.json içermeyen tamamlanmamış runlar resume adayıdır. Tamamlanmış run asla üzerine yazılmaz. Aynı ad varsa _r1, _r2 gibi ek oluşturulur.
Cell 7 - Atomic I/O
JSON, YAML, CSV, text ve torch checkpoint dosyalarını önce .tmp dosyasına yazıp fsync/read-back doğrulamasından sonra os.replace ile hedefe geçirir. Config, environment, pip freeze ve training.log burada
hazırlanır.
Cell 8 - Metadata Validation
Notebookun en kapsamlı kalite kapısıdır. Gerekli sütunları, label/split/status değerlerini, veri muhasebesini,
sample_id benzersizliğini, dosya varlığını, splitlerde iki sınıfın bulunmasını, path/hash çakışmalarını ve

mümkünse source_video/person_id izolasyonunu kontrol eder. Standardize metadata ve audit dosyaları üretir.
Cell 9 - Local SSD Cache
Eligible ROI dosyalarını /content/eyebrow_roi_cache dizinine sample_id adıyla kopyalar. Var olan cache dosyası sağlamsa tekrar kopyalamaz; bozuksa silip yeniden üretir. Kopyalama tmp -> verify -> os.replace mantığıyla yapılır.
Cell 10 - Transforms/Dataset/DataLoader
Kaş geometrisine özel ResizePadSquare tanımlanır. Train setine RandomHorizontalFlip ve hafif ColorJitter uygulanır; val/test augmentation almaz. Tüm setler ImageNet normalize edilir. EyebrowROIDataset dosyayı açar, transform uygular ve image/label/sample_id/path döndürür. Train shuffle=True, val/test shuffle=False
oluşturulur.
Cell 11 - I/O Speed Sanity
Train DataLoader'dan 5 batch okuyarak lokal cache ve loader hızının kaba sağlık kontrolünü yapar. 60 saniyeden uzunsa warning üretir; otomatik olarak eğitimi durdurmaz.
Cell 12 - Model
Torchvision pretrained EfficientNet-B0 yüklenir. Orijinal classifier kaldırılır ve Dropout(0.2) + Linear(in_features,1) ile ikili sınıflandırma head'i eklenir. Model GPU/CPU'ya taşınır ve mimari model_architecture.txt içine yazılır.
Cell 13 - Metrics
Accuracy, balanced accuracy, precision, recall, F1, specificity, ROC-AUC, average precision ve confusion matrix değerlerini üretir. Threshold seçimi validation üzerinde 0.05-0.95 arasında 181 aday için yapılır;

öncelik F1, sonra balanced accuracy, sonra 0.5'e yakınlıktır.
Cell 14 - Checkpoint + RNG Helpers
Python, NumPy, Torch, CUDA ve DataLoader generator RNG durumlarını checkpoint içine ekler. Checkpoint schema/run schema/stage name uyumluluğunu doğrular. Yükleme CPU üzerinden yapılıp model ve optimizer state mevcut cihaza taşınır.
AISC DeepFake Çalışmaları | EfficientNet-B0 Kaş ROI Kod Raporu
Nazlıcan / Deney 1 - Teknik Kod Çalışma Mantığı Raporu
Cell 15 - Safe FP32 Epoch Runner
Bir epochun train veya eval çalışmasını yürütür. Görüntü, label, logit, loss, gradient norm ve probability için
NaN/Inf kontrolü yapar. Training modunda backward, gradient clipping ve optimizer.step uygular. Tüm sample_id/path/probability/label verisini toplar ve işlenen örnek sayısını dataset uzunluğuyla doğrular.
Cell 16 - Smoke + Round-trip Tests
İki batch ile gerçek forward/backward/optimizer adımı denenir; sonrasında model ağırlıkları test öncesi haline geri yüklenir. Ardından geçici checkpoint kaydedilir, yeni model oluşturulur, checkpoint yüklenir ve
aynı mini örneklerde loss değerinin tolerans içinde birebir kaldığı kontrol edilir.
Cell 17 - Training Function
Her stage için AdamW, ReduceLROnPlateau, resume, epoch loop, train+validation değerlendirmesi, val ROC-AUC izleme, early stopping, best/last/epoch checkpoint yönetimi ve history CSV yazımını tek fonksiyonda birleştirir.
Cell 18 - Frozen Stage
model.features parametrelerinin requires_grad=False, classifier parametrelerinin True yapılmasıyla yalnız

yeni head eğitilir. frozen_backbone=True iken features eval modunda tutulduğu için BatchNorm ve feature dropout davranışı da sabitlenir.
Cell 19 - Full Fine-tuning
Stage 1 best.ckpt ağırlıkları yüklenir, tüm parametreler yeniden trainable yapılır ve 2e-5 LR ile tam ağ fine-
tuning'e geçilir. Stage 2'nin kendi uyumlu last.ckpt dosyası varsa train_stage içinde resume edilir.
Cell 20 - Validation Threshold Selection
Final fine-tune best.ckpt yüklenir. Validation setinden olasılıklar üretilir ve select_validation_threshold ile
karar eşiği seçilir. Arama tablosu ve seçilmiş validation metrikleri kaydedilir.
Cell 21 - Final Test
Test setine yalnızca final model ve validationdan gelen threshold uygulanır. Her örnek için prob_fake, pred_int, prediction, correct alanları oluşturulur; final_test_metrics.csv ve test_predictions.csv yazılır.
Cell 22 - Figures
Frozen + fine-tune history birleştirilir; loss, validation metric, test ROC, test PR, confusion matrix ve

validation threshold analysis grafikleri oluşturulur. Kaydedilen her görselin kısa kenarının en az 600 px

olması kodla doğrulanır.
Cell 23 - Reload / Inference Gate
Pretrained=False yeni bir EfficientNet-B0 örneği sıfırdan oluşturulur, best.ckpt state_dict yüklenir ve 4

test
örneği üzerinde olasılık üretir. Çıkışın 1-boyutlu, finite ve [0,1] aralığında olması zorunludur.
Cell 24 - Manifest + Summary + Complete
Runın nihai özetini yazar, RUN_COMPLETE markerı oluşturur, bütün çıktı dosyalarını dolaşarak relative path, size ve <=50 MB dosyalar için SHA-256 manifesti üretir. run_summary içindeki output_file_count güncellenir ve deney tamamlanır.
AISC DeepFake Çalışmaları | EfficientNet-B0 Kaş ROI Kod Raporu
Nazlıcan / Deney 1 - Teknik Kod Çalışma Mantığı Raporu
6. Veri Doğrulama, Muhasebe ve Leakage Kontrolleri
6.1 Zorunlu metadata şeması
Kod en az sample_id, label, split ve status sütunlarını zorunlu tutar. Ayrıca görüntü yolunu çözebilmek için
output_relative_path veya output_path alanlarından en az biri bulunmalıdır. Alan adları strip edilir; metinsel
değerler normalize edilerek label/split/status küçük harfe çevrilir.
6.2 Accounting equality
total_metadata_rows == accepted_success + skipped + error
Bu eşitlik bozulursa pipeline eğitimden önce durur. Böylece metadata içinde tanımsız veya hesaba katılmayan satırların sessizce kaybolması engellenir. SUCCESS/OK kayıtları model eğitimine adaydır; SKIPPED ve ERROR satırları audit amacıyla ayrı saklanır.
6.3 sample_id ve dosya varlığı
Eğitime alınan her accepted satırda sample_id dolu olmalı ve accepted örnekler arasında benzersiz olmalıdır. output_relative_path önceliklidir; böylece local source mount değişikliklerine karşı taşınabilirlik korunur.
Yol çözümlemesinden sonra fiziksel dosyanın gerçekten var olup olmadığı kontrol edilir.
6.4 Split kalite kapıları
☐ train, val ve test splitlerinin üçü de bulunmalıdır.
☐ Her split hem real hem fake sınıfını içermelidir.
☐ Aynı resolved ROI path farklı splitlerde bulunamaz.
☐ input_sha256 alanı varsa aynı kaynak frame hash'i farklı splitlerde bulunamaz.
☐ source_video/person_id/subject_id tam doluysa grup kümeleri birbirinden disjoint olmalıdır.
Önemli sınır: source_video/person_id/subject_id alanları yoksa veya boşsa kod kaynak kimliğini dosya adından
tahmin etmez. source_group_leakage_status = NOT_VERIFIABLE olarak raporlar.
CONFIG.allow_unverifiable_source_split=True olduğundan bu durumda warning ile eğitim devam eder.
7. Kaş ROI Ön İşleme Tasarımı
7.1 Neden doğrudan 224x224 resize yapılmıyor?
Kaş ROI görüntüleri doğal olarak yatay, geniş ve kısa bir yapıya sahiptir. Bu görüntünün doğrudan kareye esnetilmesi dikey geometrinin yapay biçimde büyümesine ve kaş kalınlığı/eğim gibi örüntülerin deformasyonuna yol açabilir. ResizePadSquare bu nedenle en-boy oranını korur.
7.2 ResizePadSquare algoritması
1. Görüntüyü RGB'ye çevirir ve width/height değerlerini kontrol eder.
2. scale = min(224 / width, 224 / height) ile görüntüyü 224x224 sınırları içine sığdıracak tek ölçek katsayısını
hesaplar.
3. BICUBIC interpolasyon ile new_width x new_height boyutuna resize eder.
4. 224x224 RGB canvas oluşturur. Padding rengi ImageNet mean değerlerinin 0-255 karşılığıdır.
5. Resize edilen kaş ROI'sini canvas ortasına yapıştırır.
6. Tensor dönüşümü ve ImageNet mean/std normalizasyonu uygulanır.
AISC DeepFake Çalışmaları | EfficientNet-B0 Kaş ROI Kod Raporu
Nazlıcan / Deney 1 - Teknik Kod Çalışma Mantığı Raporu
scale = min(size / width, size / height)
new_width = round(width * scale)
new_height = round(height * scale)
canvas = 224 x 224
ROI -> canvas center
7.3 Train ve evaluation dönüşümlerinin ayrılması
Train transform zinciri RandomHorizontalFlip(p=0.5) ve brightness/contrast=0.10 ColorJitter içerir. Bu augmentation yalnız train setine uygulanır. Validation ve testte yalnız resize+padding, ToTensor ve Normalize vardır. Bu ayrım değerlendirme verisinin yapay varyasyonla değiştirilmemesini sağlar.
8. Dataset ve DataLoader Mimarisi
EyebrowROIDataset, Pandas satırlarını PyTorch örneklerine dönüştürür. __getitem__ her örnekte training_eyebrow_path dosyasını açar, RGB'ye dönüştürür, ilgili transformu uygular ve dört alan döndürür:

image, label, sample_id ve path.
Alan İçerik Neden tutuluyor?
image 3x224x224 float tensor Model girdisi.
label float32 0/1 BCEWithLogitsLoss hedefi.
sample_id string Tahminlerin örnek bazlı izlenebilirliği.

path string Hata analizi ve prediction CSV bağlantısı.
Train DataLoader shuffle=True ve seedli torch.Generator ile oluşturulur. Validation/test shuffle=False kullanır. pin_memory yalnız CUDA aktifken açılır. drop_last=False olduğundan son eksik batch de değerlendirmeye dahildir.
Cell 10 ayrıca ilk train batchinin görüntü şeklinin tam olarak (batch, 3, 224, 224) olup olmadığını doğrular. Bu
kontrol preprocessing zincirindeki boyut hatalarını eğitime başlamadan yakalar.
9. EfficientNet-B0 Model Mimarisi
build_model fonksiyonu EfficientNet_B0_Weights.DEFAULT ile ImageNet pretrained ağırlıklarını yükler. Kod internet/ağırlık erişimi hatasında anlaşılır RuntimeError üretir. Orijinal EfficientNet-B0 classifier head'i ikili
sınıflandırma için değiştirilir:

```
model.classifier = nn.Sequential(
   nn.Dropout(p=0.2),
   nn.Linear(in_features, 1),
)
```

Tek çıkış, fake sınıfının logit skorudur. Model forward sırasında sigmoid uygulamaz; BCEWithLogitsLoss sigmoid + binary cross entropy işlemlerini sayısal olarak daha kararlı biçimde birlikte gerçekleştirir. Olasılığa
dönüşüm yalnız evaluation tarafında torch.sigmoid(logits) ile yapılır.
9.1 Frozen backbone davranışı
Stage 1'de model.features parametreleri requires_grad=False yapılır. run_epoch eğitim moduna geçtiğinde frozen_backbone=True ise model.features.eval(), model.classifier.train() çağrılır. Böylece sadece gradient
akışı değil, feature extractor içindeki BatchNorm/dropout çalışma davranışı da sabit tutulur.
9.2 Full fine-tuning davranışı
Stage 2 başlangıcında Stage 1 best.ckpt yüklenir ve bütün parametreler requires_grad=True yapılır. Bu aşamada modelin tamamı 2e-5 gibi düşük bir LR ile kaş deepfake görevine adapte edilir.
AISC DeepFake Çalışmaları | EfficientNet-B0 Kaş ROI Kod Raporu
Nazlıcan / Deney 1 - Teknik Kod Çalışma Mantığı Raporu
10. Loss, Metrikler ve Threshold Seçimi
10.1 Eğitim loss fonksiyonu
criterion = nn.BCEWithLogitsLoss() kullanılır. $y \in \{0,1\}$ ve z model logiti olmak üzere kavramsal olarak loss,
sigmoid(z) ile oluşan fake olasılığının binary cross entropy'sidir. BCEWithLogitsLoss, sigmoid işlemini içsel
olarak yaparak sayısal taşma riskini azaltır.
10.2 Hesaplanan metrikler

Metrik	Tanım	Kod açısından anlamı
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Genel doğru sınıflandırma oranı.
Balanced Accuracy	(Recall + Specificity)/2	İki sınıf performansını dengeli değerlendirir.
Precision	TP/(TP+FP)	Fake tahminlerinin ne kadarının gerçekten fake olduğu.
Recall	TP/(TP+FN)	Gerçek fake örneklerin ne kadarının yakalandığı.
F1	2PR/(P+R)	Precision ve recall harmonik ortalaması.
Specificity	TN/(TN+FP)	Real örnekleri doğru real tanıma oranı.
ROC-AUC	Threshold bağımsız	Olasılık sıralama/ayrıştırma gücü.
Average Precision	PR tabanlı	Precision-recall performans özeti.

10.3 Threshold seçiminin nasıl yapıldığı
Validation setinde 0.05 ile 0.95 arasında 181 threshold denenir. Her threshold için binary_metrics çağrılır.
Tablo F1 azalan, balanced accuracy azalan ve 0.5'e uzaklık artan sıraya göre sıralanır. İlk satır seçilen

threshold olur.
sort priority:
1) F1 -> descending
2) Balanced Accuracy-> descending
3) |threshold-0.5| -> ascending
Test izolasyonu: Threshold araması test setinde yapılmaz. Test yalnız validationdan seçilmiş threshold ile final
değerlendirmeye girer.
11. Checkpoint, Resume ve RNG Yönetimi
11.1 Checkpoint içinde ne saklanıyor?

Alan	Amaç
checkpoint_schema_version	Checkpoint yapısının sürümü.
run_schema_version	Run yapısının notebook sürümüyle uyumu.
epoch	Son tamamlanan epoch.

stage_name frozen / finetune / smoke.
model_state_dict Model ağırlıkları.
optimizer_state_dict AdamW durumu.

Alan Amaç
scheduler_state_dict ReduceLROnPlateau durumu.
scaler_state_dict FP32 olduğu için None; şema alanı korunur.
best_metric_score O ana kadarki en iyi validation monitor.
history Epoch bazlı metrik geçmişi.
config Çalıştırılan Config snapshotı.
rng_state Python/NumPy/Torch/CUDA/DataLoader RNG durumu.
Checkpoint önce geçici .tmp dosyasına torch.save ile yazılır, CPU üzerinde tekrar torch.load ile açılır ve
zorunlu anahtarlar doğrulanır. Son adımda os.replace kullanılır. Böylece yarım yazılmış checkpointin geçerli
best.ckpt/last.ckpt adını alması engellenir.
11.2 Resume politikası
Notebook önce run düzeyinde yalnız uyumlu, RUN_COMPLETE markerı bulunmayan bir runı seçer. Stage düzeyinde last.ckpt checkpoint_schema_version, run_schema_version ve stage_name ile doğrulanır. Resume sırasında model, optimizer, scheduler, history, best_score ve RNG durumları restore edilir. Böylece
yalnız model ağırlıkları değil eğitim durumu da sürdürülür.
12. Eğitim Döngüsü ve İki Aşamalı Transfer Learning
12.1 run_epoch içindeki bir batchin yaşam döngüsü
1. Batch görüntü ve etiketleri DEVICE'a taşınır.
2. Images ve labels finite mi kontrol edilir.
3. Training ise optimizer gradientleri sıfırlanır.
4. Model FP32 logit üretir; logit finite kontrol edilir.
5. BCEWithLogitsLoss hesaplanır ve finite kontrol edilir.
6. Training ise backward yapılır; tüm parameter.grad tensorları finite mi kontrol edilir.
7. Global gradient norm 1.0 üst sınırında clip edilir; norm finite olmalıdır.
8. optimizer.step uygulanır.
9. Logitlerden sigmoid ile probability üretilir; probability finite kontrol edilir.
10. Loss, label, probability, sample_id ve path biriktirilir.
11. Epoch sonunda processed sayısı tam dataset uzunluğuna eşit olmalıdır.

12.2 train_stage eğitim kontrolü
train_stage, her stage için yalnız requires_grad=True parametreleri AdamW optimizera verir. ReduceLROnPlateau scheduler mode="max", factor=0.5, patience=2 ile validation monitor düşmediğinde LR'yi azaltır. Ana monitor validation ROC-AUC'dir; ROC-AUC finite değilse F1 fallback olarak kullanılır. Her epoch sonunda last.ckpt ve epoch_NNN.ckpt kaydedilir. Yalnız son üç epoch backup tutulur. Monitor yeni best_score değerini aşarsa ayrıca best.ckpt güncellenir. İyileşme 4 epoch boyunca gelmezse early stopping devreye girer.
AISC DeepFake Çalışmaları | EfficientNet-B0 Kaş ROI Kod Raporu
Nazlıcan / Deney 1 - Teknik Kod Çalışma Mantığı Raporu
12.3 Stage 1 ve Stage 2 arasındaki ilişki
Stage Trainable bölüm LR Başlangıç ağırlığı Amaç
Frozen Yalnız classifier 1e-3 ImageNet pretrained EfficientNet-B0 Yeni ikili head'i kaş görevine hızlıca adapte etmek.
Fine-tune Tüm ağ 2e-5 Stage 1 best.ckpt Feature extractorı düşük LR ile kaş deepfake örüntülerine uyarlamak.
13. Kalite Kapıları ve Hata Yönetimi
Kalite kapısı Kontrol Başarısızlık davranışı
local source gate LocalSource ve proje klasörleri gerçekten mevcut mu? Yoksa eğitim başlamaz.
Metadata schema Zorunlu sütun ve değerler doğru mu? Hata ile durur.
Accounting Tüm satırlar success/skipped/error olarak muhasebeleşti mi? Hata ile durur.
Sample ID Accepted sample_id dolu ve benzersiz mi? Hata ile durur.
File existence Eligible görseller diskte var mı? Eksikler eğitim dışında kalır; sayı raporlanır.

Split classes Her split real + fake içeriyor mu? Hata ile durur.
Path leakage Aynı ROI path farklı splitte mi? Hata ile durur.
Hash leakage Aynı input SHA-256 farklı splitte mi? Hata ile durur.
Source group source_video/person_id disjoint mi? Varsa hata; yoksa NOT_VERIFIABLE
warning.
Cache verify Kopyalanan görüntü PIL verify geçiyor
mu?
Hata ile durur.
Batch shape Girdi 3x224x224 mü? Hata ile durur.
I/O speed 5 batch 60 sn üstü mü? Warning; eğitim devam eder.
Smoke test 2 batch forward/backward çalışıyor mu? Hata ile durur.
Checkpoint roundtrip Save-load sonrası loss aynı mı? Hata ile durur.
Numerical gate Images/labels/logits/loss/grad/prob finite
mi?
FloatingPointError ile durur.
Epoch accounting Processed == dataset size mı? Hata ile durur.
Figure quality Kısa kenar >=600 px mi? Hata ile durur.
Inference reload best.ckpt yeni modelde inference veriyor
mu?
Hata ile durur.
Kod özellikle eğitim sırasında oluşabilecek sessiz sayısal bozulmaları kabul etmez. NaN/Inf görüntü,
logit,
loss, gradient veya probability görüldüğünde epochun geri kalanına devam etmek yerine süreci durdurur.

14. Final Test, Tahmin Kayıtları ve Görselleştirme
14.1 Final test CSV yapısı
Test çıktı tablosu yalnız toplu metrikleri değil, her örneğin tahminini de saklar. test_predictions.csv
alanları:
sample_id, path, label_int, label, prob_fake, threshold, pred_int, prediction ve correct. Bu yapı yanlış
sınıflanan örnekleri daha sonra görsel olarak incelemeye uygundur.
14.2 Üretilen grafikler
Dosya İçerik
training_validation_loss_curve.png Frozen ve fine-tune aşamalarının birleşik train/validation loss
eğrisi.
validation_metrics_curve.png Validation ROC-AUC, F1 ve balanced accuracy gelişimi.
test_roc_curve.png Test ROC eğrisi ve AUC.
test_precision_recall_curve.png Test precision-recall eğrisi ve Average Precision.
test_confusion_matrix.png Validationdan seçilmiş threshold ile final test confusion matrix.
validation_threshold_analysis.png Thresholda göre validation F1, balanced accuracy ve specificity
değişimi.
Grafiklerin başlık, eksen ve legend metinleri İngilizcedir. save_figure fonksiyonu 150 DPI ile kaydeder
ve PIL
ile kısa kenar >=600 px kalite kapısını uygular.
15. Deney Çıktı Dosyaları ve Manifest
Her çalışma Sonuçlar/<run_id> altında izole edilir. Notebook tarafından oluşturulan ana yapı aşağıdaki
gibidir:
<run_id>/
```
├── checkpoints/
│   ├── frozen/{best.ckpt,last.ckpt,epoch_*.ckpt}
│   └── finetune/{best.ckpt,last.ckpt,epoch_*.ckpt}
├── logs/training.log
├── metrics/
├── predictions/test_predictions.csv
├── figures/*.png
├── artifacts/
├── config_resolved.yaml
├── environment.json
├── requirements_lock.txt
├── data_accounting.json
├── run_schema.json
├── run_summary.json
├── RUN_COMPLETE.json
└── output_manifest.csv
```
build_output_manifest fonksiyonu run dizini altındaki her dosyayı tarar, göreli yolunu ve boyutunu yazar.
50
MB veya daha küçük dosyalarda SHA-256 hesaplar; daha büyük dosyalarda hash alanını boş bırakır.
Manifest kendisini recursive olarak hashlemez ve .tmp dosyalarını dışlar.
16. Tekrar Üretilebilirlik ve Sürdürülebilirlik

Notebook tekrar üretilebilirliği üç katmanda ele alır: konfigürasyonun kalıcılaştırılması, RNG seed/durumlarının yönetilmesi ve çevre bilgisinin kaydedilmesi. seed=42 Python, NumPy, Torch, CUDA ve

DataLoader generator için uygulanır. Deterministic algoritmalar tercih edilir; desteklenmeyen operasyonlarda
warn_only kullanıldığı için çalışma tamamen bloklanmaz.
environment.json Python/PyTorch/device/GPU/platform bilgilerini, requirements_lock.txt ise pip freeze çıktısını saklar. pip freeze komutu başarısız olursa kod deneyin tamamını durdurmak yerine hata metnini requirements_lock.txt içine yazar; yani bağımlılık snapshotının başarısızlığı görünür hale getirilir.
Resume açısından last.ckpt model+optimizer+scheduler+history+RNG durumlarını birlikte taşır. Böylece donanım/runtime kesintisi sonrası aynı stagein kaldığı epoch sonrasından devam edilmesi hedeflenir.
Tamamlanmış bir runın RUN_COMPLETE markerı, yanlışlıkla üzerine yazmayı önleyen bir sınır görevi görür.

17. Kodun Güçlü Yönleri

☐ Kaş ROI geometrisine özgü aspect-ratio koruyan preprocessing tasarımı.
☐ Input Kaş klasörünü değiştirmeden sonuçları run bazlı ayrı dizinlere yazması.
☐ Metadata muhasebesi ve accepted/audit ayrımının açık olması.
☐ Exact path ve source-frame SHA-256 tabanlı split leakage kontrolleri.
☐ Gerçek source_video bilgisi yoksa bunu uydurmaması ve NOT_VERIFIABLE olarak raporlaması.
☐ local source I/O darboğazını azaltmak için doğrulamalı lokal SSD cache.
☐ Pretrained model + frozen head warm-up + düşük LR full fine-tuning tasarımı.
☐ Test setini threshold/model seçimine sokmaması.
☐ Atomik checkpoint ve read-back doğrulaması.
☐ Optimizer, scheduler ve RNG dahil tam resume state saklaması.
☐ NaN/Inf ve gradient kalite kapılarının batch düzeyinde aktif olması.
☐ İki batch smoke test ve bağımsız checkpoint round-trip testi.
☐ Final best checkpointin sıfırdan modele yüklenerek inference test edilmesi.
☐ Run sonunda manifest, hash, config, environment ve completion marker üretmesi.

18. Kodun Sınırları ve Dikkat Edilecek Noktalar

Konu	Açıklama
Video/kişi izolasyonu	source_video/person_id yoksa yalnız path ve input SHA-256 izolasyonu doğrulanabilir. Gerçek video bazlı split kanıtı oluşmaz.
allow_unverifiable_source_split=True	Mevcut ayarda source grup doğrulanamasa da eğitim warning ile devam eder. Tam katı politika için False yapılmalıdır.
Upstream split korunuyor	Notebook train/val/testi yeniden üretmez. Dolayısıyla upstream splitin nasıl oluşturulduğu bu kod tarafından geriye dönük kanıtlanamaz.
Tek model/tek seed	Kod bu run içinde EfficientNet-B0 ve seed 42 kullanır. Çoklu seed veya cross-validation otomatik değildir.
Augmentation kapsamı	Yalnız horizontal flip ve hafif brightness/contrast vardır. Daha geniş augmentation politikası kodda bulunmaz.
Class weighting yok	BCEWithLogitsLoss varsayılan ağırlıksız kullanılır; pos_weight veya sampler uygulanmaz.
Threshold hedefi	Seçim önceliği F1'dir. F1'i optimize eden threshold yüksek recall fakat düşük specificity gibi dengesiz bir çalışma noktası seçebilir.
FP32	Mixed precision kullanılmaz. Bu sayısal kararlılığı sadeleştirir fakat eğitim hızı/bellek açısından AMP avantajlarını kullanmaz.
AISC DeepFake Çalışmaları | EfficientNet-B0 Kaş ROI Kod Raporu
Nazlıcan / Deney 1 - Teknik Kod Çalışma Mantığı Raporu

Konu	Açıklama
I/O speed gate	warning-only 60 saniye üstü loader testi eğitimi bloklamaz; yalnız kullanıcıyı uyarır.
Manifest hash sınırı	50 MB üstü çıktı dosyaları SHA-256 manifest hash'i almaz.
Raporlama önerisi:	Bu notebookla elde edilen bilimsel sonuçlar raporlanırken source-group leakage durumunun NOT_VERIFIABLE olduğu açıkça belirtilmeli; upstream source_video/kişi ID tamamlandığında deney aynı pipeline ile yeniden doğrulanabilir.

19. Sonuç

EfficientNet_B0_Eyebrow_Nazlican_FINAL_STABLE notebooku, basit bir model eğitim hücreleri dizisinden daha kapsamlı bir deney yürütme altyapısıdır. Kod veri girişinden başlayıp final modelin yeniden yüklenebilirliğine kadar birçok kalite kapısını aynı workflow içinde birleştirir. Kaş ROI'lerinin geometrik
karakterini koruyan preprocessing, iki aşamalı transfer learning, validation tabanlı threshold seçimi ve atomik
checkpoint yaklaşımı notebookun temel teknik omurgasını oluşturur.
Akışın en önemli araştırma ilkesi, test setinin model/threshold seçimine dahil edilmemesi ve mevcut

metadata yetersizse kaynak video kimliğinin uydurulmamasıdır. Bu sayede notebook, bilmediği veri provenance bilgisini gizlemek yerine açık bir NOT_VERIFIABLE durumu üretir.

Kodun çalışma mantığı özetle şu zincirle ifade edilebilir: doğrula -> cache et -> geometrisini koruyarak dönüştür -> pretrained modeli hazırla -> kalite kapılarını geç -> frozen eğit -> fine-tune et -> validation
threshold seç -> final test yap -> yeniden yüklenebilirliği kanıtla -> tüm artefaktları manifestle ve runı
tamamla.

Ek A. Hücre Haritası

Cell	Başlık	Temel görev
0	Notebook açıklaması	Amaç, yollar, çıktı standardı, eğitim politikası.
1	Imports	Kütüphaneler ve ortam bilgisi.
2	Configuration	Deney parametreleri.
3	Reproducibility	Seed ve deterministic ayarlar.
4	local source	local source package mount.
5	Paths	Kaş ve Sonuçlar yol doğrulaması.
6	Run manager	Run ID ve resume.
7	Atomic I/O	Güvenli kayıt + env/config/log.
8	Metadata	Şema, accounting, leakage, audit.
9	Cache	Lokal SSD image cache.
10	Data pipeline	Preprocessing, Dataset, DataLoader.
11	Speed test	5-batch I/O sanity.
12	Model	EfficientNet-B0 binary head.
13	Metrics	Metrikler + threshold arama.
14	Checkpoint/RNG	Save/load/resume state.

Cell	Başlık	Temel görev
15	Epoch runner	FP32 train/eval batch döngüsü.
16	Smoke gates	Forward/backward + checkpoint roundtrip.
17	Train stage	Optimizer/scheduler/early stopping/checkpoints.
18	Frozen stage	Classifier-only eğitim.
19	Fine-tune stage	Tüm ağ eğitimi.
20	Validation threshold	Best checkpoint + threshold selection.
21	Final test	Final metrik ve prediction CSV.
22	Figures	6 standart grafik.
23	Inference reload	Best model reload kalite kapısı.
24	Finalization	Summary, complete marker, manifest.

AISC DeepFake Çalışmaları | EfficientNet-B0 Kaş ROI Kod Raporu
Nazlıcan / Deney 1 - Teknik Kod Çalışma Mantığı Raporu

Ek B. Kritik Fonksiyonlar ve Görevleri

Fonksiyon	Görevi
seed_everything	Tüm RNG seed ve deterministic ayarları.
is_compatible_incomplete_run	Resume için tamamlanmamış run uyumluluğu.
atomic_write_bytes/text/json/yaml/csv	Dosyaları tmp + fsync + replace ile atomik yazma.
atomic_torch_save	Checkpoint save + read-back schema doğrulama.
sha256_file	Dosya bütünlüğü hash'i.
resolve_eyebrow_path	Relative/absolute ROI yolunu fiziksel Path'e dönüştürme.
cache_one_image	ROI'yi lokal SSD'ye güvenli/verify edilmiş kopyalama.
ResizePadSquare	Kaş geometrisini koruyarak kare canvas hazırlama.
EyebrowROIDataset.__getitem__	Görüntü okuma, transform, label/sample_id/path döndürme.
build_model	Pretrained EfficientNet-B0 + binary head.
binary_metrics	Threshold tabanlı binary metrik hesaplama.
select_validation_threshold	Validation threshold taraması ve seçim kriteri.
get_rng_state / set_rng_state	Checkpoint için RNG snapshot/restore.
save_checkpoint / load_checkpoint	Tam eğitim state kayıt/yükleme.
run_epoch	FP32 train/eval ve numerical quality gates.
model_smoke_test	2 batch forward/backward çalışma testi.
checkpoint_roundtrip_test	Kaydet-yükle sonrası loss eşitliği testi.
train_stage	Stage eğitim orchestration.
save_figure	Grafik save + min 600px doğrulama.
inference_reload_test	best.ckpt bağımsız inference testi.
build_output_manifest	Run artefakt listesi, size ve SHA-256 manifesti.

Kaynak ve İnceleme Notu

Bu raporun teknik içeriği doğrudan "EfficientNet_B0_Eyebrow_Nazlican_FINAL_STABLE (1).ipynb" notebookundaki 25 hücrenin kod akışı incelenerek hazırlanmıştır. Sonuç performansı yorumundan ziyade kodun ne yaptığı, hangi sırayla yaptığı, hangi dosyaları ürettiği ve hangi kalite kapılarını uyguladığı

açıklanmıştır.